

AI-Enhanced CRM & Ticket Management System

Course	Artificial Intelligence (CS-4XX)
Assignment Type	Major Project Assignment
Marks	100 Marks
Duration	One (1) Week
Team Size	Individual or Groups of 2-3 Students
Submission Mode	GitHub Repository + Report + Live Demo
Late Submission	5 marks deducted per day after deadline

This assignment challenges you to design and develop an AI-enhanced CRM (Customer Relationship Management) and Ticket Management System, integrated with a real-world messaging platform. You are free to choose any programming language, framework, or deployment approach. Creativity, thoughtful AI integration, and practical implementation are the hallmarks of an excellent submission.

Read this document carefully in its entirety before beginning. Good luck, and enjoy building!

1. Assignment Overview

Customer Relationship Management (CRM) systems are at the core of how modern businesses manage their customer interactions, track service requests, and maintain relationships over time. When augmented with Artificial Intelligence, these systems become powerful tools that can automate repetitive work, predict problems before they escalate, and deliver superior customer experiences.

In this assignment, you will build a functional CRM and Ticket Management System that incorporates AI-powered features and integrates with at least one external messaging platform. You are not expected to build an enterprise-grade product you are expected to demonstrate your ability to design, implement, and connect real systems in creative and practical ways. Your system should allow agents (support staff) to manage customer records, create and resolve support tickets, communicate updates, and leverage AI to make smarter decisions. Think of it as a mini-Zendesk or mini-Freshdesk but one you built yourself.

2. Learning Objectives

By the end of this assignment, students will be able to:

1. Design and implement a multi-module software system with a database backend
2. Integrate Artificial Intelligence APIs or libraries into a real-world application
3. Connect an application to an external messaging or communication platform
4. Apply user authentication and role-based access control in a web or desktop app
5. Design a relational or document-based database schema for a CRM use case
6. Produce technical documentation explaining system architecture and design decisions
7. Demonstrate a working system and communicate design choices clearly in a viva
8. Develop practical problem-solving skills through open-ended, self-directed development

3. Problem Statement

A mid-sized technology company, TechServe Solutions, receives hundreds of customer support queries per week via email, messaging apps, and phone calls. Their current process is entirely manual agents copy-paste messages into spreadsheets, assign tickets by hand, and follow up via personal chat accounts. The result is missed tickets, slow response times, frustrated customers, and overwhelmed support staff.

TechServe Solutions has hired your team to build a Minimum Viable Product (MVP) of an AI-Enhanced CRM and Ticket Management System. This system should:

1. Provide a central dashboard for managing customers and tickets
2. Allow agents to create, update, and resolve support tickets
3. Automatically categorize or prioritize tickets using AI
4. Send real-time ticket notifications to agents through a messaging platform
5. Maintain a log of all communications and ticket activities
6. Provide a simple, usable interface for both agents and managers

You are TechServe Solutions' development team. Your task is to build this MVP within one week.

4. Functional Requirements

The following functional requirements must be met. These define the minimum acceptable behavior of the system.

1.1 User Authentication & Authorization

1. The system must support user login and logout with secure session management
2. At least two roles must be implemented: Agent and Manager (Admin)
3. Agents can create and manage tickets; Managers can manage all data and view reports
4. Passwords must be stored securely (hashed; no plain text)
5. Role-based access control must prevent unauthorized access to protected routes

1.2 Customer Management

1. Ability to add, view, edit, and delete customer records
2. Each customer record must include: Full Name, Email, Phone Number, Company, and Date Added
3. Ability to search and filter customers by name, email, or company
4. View a customer's full ticket history from their profile page

4.3 Ticket Management

1. Ability to create a new support ticket linked to a customer
2. Each ticket must include: Title, Description, Status, Priority, Category, Agent Assigned, and Date Created
3. Ticket statuses must include at least: Open, In Progress, Resolved, and Closed
4. Ticket priorities must include at least: Low, Medium, High, and Critical
5. Agents must be able to update ticket status and add internal notes
6. Ticket history/changelog must be maintained to show all status changes and comments
7. Tickets must be filterable by status, priority, agent, and date

4.4 Dashboard

1. A central dashboard must display key summary statistics (e.g., total open tickets, resolved today, critical tickets)
2. Visual indicators such as charts, graphs, or badge counts are encouraged
3. Managers must be able to see agent-wise ticket workloads
4. Agents must be able to see their own assigned tickets at a glance

4.5 Communication & Activity Log

1. Each ticket must have an activity/comment thread where agents can log updates
2. Timestamps and author names must appear on all log entries
3. Communication history per customer must be viewable from the customer profile

4.6 Messaging Platform Integration

1. The system must integrate with at least ONE messaging platform (see Section 7)
2. At a minimum, automatic notifications must be sent when: a new ticket is created, a ticket is resolved, or a ticket is escalated to Critical
3. Bonus: Allow customer queries to be received directly through the messaging platform

4.7 AI-Powered Feature(s)

5. The system must include at least ONE AI-powered feature (see Section 8 for ideas)
6. The AI feature must be genuinely useful not a placeholder or decoration
7. The AI integration must be clearly described in the project report

5. AI Integration Ideas

You must implement at least ONE of the following AI-powered features. You are encouraged to implement more for bonus marks. Use AI APIs (such as OpenAI, Gemini, Claude, Cohere, Hugging Face), fine-tuned models, or open-source libraries the choice is entirely yours.

Suggested AI Features (choose at least one)

1. Auto Ticket Categorization: Automatically classify a ticket into a category (e.g., Billing, Technical, Account, Shipping) when it is submitted, using NLP or an LLM API
2. Sentiment Analysis: Analyze the sentiment of the ticket description or customer message (Positive, Neutral, Negative, Frustrated) to help agents prioritize emotionally charged tickets
3. AI-Generated Response Suggestions: Suggest a draft reply to an agent when they open a ticket, based on the ticket content and similar past tickets
4. Ticket Urgency Prediction: Use ML or LLM to predict whether an incoming ticket is urgent, even if not marked as Critical, based on language cues
5. Smart Customer Prioritization: Use AI to score or rank customers by satisfaction risk, based on their ticket history and sentiment trends
6. FAQ / Knowledge Base Chatbot: Provide a simple in-app chatbot that can answer common questions before a customer submits a ticket
7. AI Conversation Summary: When a ticket is resolved, automatically generate a summary of the conversation and save it to the ticket record
8. Auto-Tagging: Use keyword extraction or NLP to automatically assign relevant tags to tickets (e.g., 'refund', 'login issue', 'hardware')

You are not limited to this list. If you have a creative AI idea not listed here, implement it and document it well innovation is rewarded.

6. Messaging Integration Ideas

Your system must connect to at least one messaging or communication platform. Below are suggested platforms and ideas for integration. You may use official APIs, community APIs, webhooks, bots, or open-source SDKs.

Platform	Integration Ideas
Telegram	Telegram Bot: send ticket notifications, receive customer queries, allow agents to update ticket status via bot commands
Discord	Discord Webhook or Bot: post ticket alerts to a channel, create threads per ticket, allow team commands
Slack	Slack Incoming Webhooks or Bolt SDK: notify a #tickets channel, interactive message buttons for quick ticket actions
Email	SMTP / SendGrid / Mailgun: send automated email alerts for ticket creation, resolution, and escalation
WhatsApp	Twilio WhatsApp API or WPPConnect (community): send WhatsApp messages for ticket updates to customers
Matrix	Matrix Client SDK or Element: integrate with an open-source federated messaging network
Mattermost / Rocket.Chat	Webhooks or REST APIs for self-hosted open-source messaging platforms

At a minimum, your integration should send real notifications to real accounts (not mocked). Screenshots of received messages must be included in your report and demonstration.

7. Technology Options

You are completely free to choose your technology stack. There is no penalty for any valid choice. The following are non-exhaustive examples to help you get started.

Layer	Examples (use any, or your own preference)
Frontend	React, Vue.js, Angular, HTML/CSS/JS, Flutter (Web), Next.js, Svelte, Blade (Laravel), Jinja2 (Django)
Backend	Node.js/Express, Python/FastAPI, Python/Django, PHP/Laravel, Java/Spring Boot, C#/.NET, Ruby on Rails, Go/Gin
Database	PostgreSQL, MySQL, MongoDB, SQLite, Firebase Firestore, Supabase, PlanetScale, MariaDB
AI / NLP	OpenAI API, Google Gemini API, Anthropic Claude API, Cohere, Hugging Face Transformers, spaCy, NLTK, Scikit-learn
Authentication	JWT, Firebase Auth, Passport.js, Laravel Sanctum, Django Auth, ASP.NET Identity, NextAuth.js
Deployment	Localhost, Vercel, Render, Railway, Heroku, AWS Free Tier, Azure Student, Docker (local or cloud)

You may also use no-code or low-code integrations (such as Zapier, Make/Integromat, or n8n) for messaging provided you document them thoroughly and they produce real, working results.

8. System Modules

Your system should be organized into the following logical modules. Implementation details are left to you.

#	Module	Description
M1	Auth Module	Login, logout, role management, session/token handling
M2	Customer Module	CRUD for customer records, search/filter, customer profile view
M3	Ticket Module	Ticket creation, assignment, status updates, comment thread, history log
M4	Dashboard Module	Metrics overview, agent workloads, recent tickets, filters
M5	AI Module	AI feature(s) integrated into the workflow (categorization, sentiment, suggestions, etc.)
M6	Messaging Module	Integration with external platform: notifications, alerts, or two-way communication
M7	Reports Module	(Optional but encouraged) Basic reports on tickets by category, agent performance, resolution time

9. Sample User Roles & Example Workflow

9.1 Sample User Roles

Role	Permissions & Responsibilities
Manager / Admin	Full access to all modules. Can create/delete agents, view all tickets across all agents, generate reports, configure system settings
Support Agent	Can create and manage tickets assigned to them, view their own customer list, add comments, and update ticket status. Cannot delete other agents' records
Customer (Optional)	(Bonus) May submit queries through the messaging platform or a customer portal; receives automated updates when their ticket is resolved

9.2 Example Ticket Lifecycle

The following illustrates a typical ticket from creation to resolution. Your system should support this flow:

Step	Action	System Behaviour
1	Customer query received	Agent creates a new ticket for the customer. AI auto-categorizes it as 'Billing'. Telegram notification sent to agents channel.
2	Ticket assigned	Manager assigns ticket to Agent Sarah. Status changes to 'In Progress'. Agent notified.

3	AI response suggestion	Agent Sarah opens the ticket. AI suggests a draft response based on the issue description.
4	Agent comments	Sarah edits the suggestion and posts it as a comment. Activity log updated with timestamp.
5	Escalation	Ticket sentiment score drops (customer frustrated). System auto-escalates to 'Critical'. Alert sent via messaging platform.
6	Resolution	Issue resolved. Status updated to 'Resolved'. AI generates a summary of the conversation. Customer notified via Telegram/Email.
7	Closure	Manager reviews and closes the ticket. Resolution summary saved. Dashboard metrics updated.

10. Suggested Database Entities

The following are suggested data entities for your system. You are free to extend, modify, or restructure them. Document your final schema in the report.

Users / Agents

9. id, name, email, password_hash, role (agent/manager), created_at, is_active

Customers

10. id, full_name, email, phone, company, notes, created_at, assigned_agent_id

Tickets

11. id, title, description, status, priority, category, customer_id, assigned_agent_id, created_at, updated_at, resolved_at
 12. ai_category (auto-filled), ai_sentiment_score, ai_summary (optional)

Ticket Comments / Activity Log

13. id, ticket_id, agent_id, message, is_internal (true/false — internal notes vs. customer-facing), created_at

Notifications / Message Log

14. id, ticket_id, platform (telegram/slack/email/etc.), message, status (sent/failed), sent_at

Categories & Tags (Optional)

15. id, name, color_code — linked to tickets via a junction table or direct foreign key

11. Minimum Requirements

To pass this assignment, your submission must meet all of the following minimum requirements:

	Minimum Requirement
MR-1	Working user login with at least two roles (Agent, Manager)
MR-2	Ability to create, view, update, and delete customer records
MR-3	Ability to create, update (status/priority), assign, and view support tickets
MR-4	Comment/activity log on tickets with timestamps
MR-5	A working dashboard with at least three summary statistics or visual indicators
MR-6	At least one functional AI feature that is genuinely triggered and produces output within the system
MR-7	At least one working messaging platform integration that sends real messages to a real account/channel
MR-8	A database with persistent data storage (no in-memory only solutions)
MR-9	A project report (see Section 17) of minimum 6 pages
MR-10	A GitHub repository with a README that includes setup instructions

12. Bonus Features

The following features are not required but will earn bonus marks (up to 10 additional marks). Bonus marks can compensate for weaknesses in other areas.

Bonus Feature Options (choose any)

1. Customer Portal: A separate interface where customers can submit and track their own tickets without agent involvement
2. Two-Way Messaging: Allow customers to reply via messaging platform and have the message appear as a ticket comment
3. Multi-Platform Integration: Integrate with two or more messaging platforms simultaneously
4. Advanced Analytics: Charts showing ticket resolution time trends, agent efficiency, category breakdown, and customer satisfaction scores
5. AI Escalation Engine: Automatically change a ticket's priority to Critical based on AI-detected urgency without manual intervention

6. Knowledge Base: A searchable FAQ page where resolved tickets can be promoted to knowledge base articles
7. Real-Time Updates: Live ticket status changes visible without page refresh (using WebSockets or polling)
8. Mobile Responsive UI: A fully responsive interface that works well on mobile devices
9. Docker Deployment: A Docker Compose setup that allows the entire system to be launched with a single command
10. Unit Tests: A basic test suite covering at least three core system functions

13. Hints & Guidance

The following guidance is provided to help you get started. These are suggestions, not instructions. You are free to deviate from any of these paths.

13.1 Getting Started Quickly

1. Start with user authentication it is the foundation of everything else
2. Build the database schema before writing any UI code
3. Build and test the Customer and Ticket modules before integrating AI or messaging
4. Use a framework you are already comfortable with this is not the time to learn a new language from scratch
5. Use environment variables (.env files) to store API keys never hard-code them

13.2 AI Integration Tips

1. For AI features, start with an LLM API (e.g., OpenAI GPT-4o-mini or Gemini Flash) they are the easiest to integrate via simple HTTP calls
2. Prompt engineering is your friend: a well-crafted system prompt can produce categorization, sentiment analysis, and response suggestions without any model training
3. Keep your AI calls asynchronous so they do not block your main application
4. Always handle AI API errors gracefully show a fallback message if the AI is unavailable
5. Store AI-generated outputs in the database so users can see them without re-calling the API

13.3 Messaging Integration Tips

1. Telegram is the easiest to start with: BotFather setup takes under 5 minutes and the API is well-documented
2. For Discord, create a server, add a webhook, and send HTTP POST requests no bot required for basic notifications
3. For Email, use a service like SendGrid, Mailgun, or Resend they offer free tiers and simple APIs
4. Test your messaging integration with a dedicated test channel before connecting it to your production system
5. Consider using a background job/queue for sending messages so they do not delay the main request cycle

13.4 Possible Simple Implementation Path (Example Only)

This is one possible path you are not required to follow it:

1. Backend: FastAPI (Python) with PostgreSQL database
7. Frontend: Simple HTML/CSS/JS or React
8. Auth: JWT tokens stored in localStorage
9. AI: OpenAI API for ticket categorization (prompt: 'Given this ticket description, return the category as JSON')
10. Messaging: Telegram bot using the python-telegram-bot library for ticket notifications
11. Deployment: Run locally, or push to Render.com for free hosting

There are dozens of other valid paths. Choose what works for your team.

14. Submission Requirements

All submissions must be made by the deadline indicated in Section 20 (Timeline). Late submissions will be penalized 5 marks per day.

14.1 What to Submit

1. GitHub Repository URL: A public (or instructor-shared) repository containing all source code
2. README File: Must include project title, team members, setup instructions, .env variable descriptions, and screenshots
3. Project Report: PDF format, minimum 6 pages (see Section 17 for full requirements)
4. Video Demo: A 3-5 minute screen recording demonstrating all core features (include messaging and AI features in the demo)
5. Live Demo URL (if deployed): Optional but encouraged submit the URL of your deployed application

14.2 Submission Method

1. Submit all materials via the course Learning Management System (LMS) or the email/link provided by your instructor
2. Use the naming format: GroupID_CRMProject_Submission.zip (e.g., G03_CRMProject_Submission.zip)
3. Ensure the repository is accessible private repos must grant access to the instructor account before submission

14.3 Checklist Before Submission

Done?	Item
☐	GitHub repository is public / shared with instructor
☐	README includes setup instructions and environment variable descriptions
☐	All 10 minimum requirements (MR-1 through MR-10) have been met
☐	Project report is complete and in PDF format

☐	Video demo is recorded and uploaded
☐	Team registration was submitted on time
☐	No API keys or secrets are committed to the repository (use .env files)
☐	All group members' names and student IDs appear on the report cover page

15. Evaluation Rubric

Your project will be evaluated across the following nine criteria, totaling 100 marks. Bonus marks of up to 10 additional marks are available for bonus features (Section 12).

Criterion	Marks	What Evaluators Look For
1. Core Functionality	25	All CRUD operations work (customers, tickets). Status/priority updates work. Dashboard shows real data. Role-based access enforced.
2. AI Integration	20	AI feature is functional and triggered within the system. Output is meaningful and used in the workflow. Integration is well-documented.
3. Messaging Integration	15	Real messages delivered to a real channel or account. Notifications triggered on correct events. Evidence shown in demo and report.
4. Database Design	10	Schema is logical, normalized, and supports all required features. Relationships are correct. Schema is documented in the report.
5. UI / UX Quality	10	Interface is clean, navigable, and functional. Forms have validation. Ticket and customer pages are usable without confusion.
6. Code Quality	7	Code is organized into modules. Functions are appropriately sized. No obvious security issues (e.g., plain-text passwords). README is complete.
7. Documentation / Report	7	Report covers all required sections. Architecture diagram is present. AI and messaging integration are clearly explained.
8. Innovation & Creativity	4	Project shows creative thinking — novel AI use, unique UX approach, unexpected integration, or extension beyond the minimum requirements.
9. Viva / Presentation	2	Student can explain design decisions, answer questions about the codebase, and demonstrate the system confidently.
TOTAL	100	Bonus features can earn up to 10 additional marks

16. Report Requirements

A project report must be submitted in PDF format. The report should be professional, clear, and follow the structure below.

#	Section	Required Content
1	Cover Page	Project title, course name, team members (names + student IDs), submission date
2	Project Overview	Brief description of what was built, key decisions made, and what differentiates your system
3	System Architecture	Architecture diagram (hand-drawn or digital). Description of each component and how they interact
4	Database Schema	ER diagram or table descriptions showing entities, attributes, and relationships
5	AI Integration	What AI feature(s) you built, why you chose them, what API/library you used, example inputs/outputs, and how it integrates into the workflow
6	Messaging Integration	What platform you chose, how it is connected, sample screenshots of real notifications received, and any challenges encountered
7	Feature Screenshots	Screenshots of all major screens: login, dashboard, customer list, ticket detail, AI output, messaging notification
8	Challenges & Learnings	What problems did you face? How did you solve them? What would you do differently?
9	Work Division (Groups)	Table showing each member and their modules/contributions. Required for group submissions.
10	References	All APIs, libraries, tutorials, and resources used. Use any consistent citation format (APA, IEEE, etc.)

17. Timeline (1-Week Schedule)

Day(s)	Milestone	Suggested Tasks
Day 1	Planning & Setup	Register team. Choose tech stack. Design database schema. Set up project structure and GitHub repo. Create .env file structure.
Day 2	Core Backend	Implement authentication (login/roles). Build Customer and Ticket CRUD APIs. Test with Postman or similar tool.
Day 3	Frontend UI	Build login screen, dashboard, customer list, ticket detail view, and ticket creation form. Connect to backend.

Day 4	AI Integration	Implement chosen AI feature(s). Test with real ticket data. Handle errors gracefully. Store AI output in DB.
Day 5	Messaging Integration	Set up messaging platform bot/webhook. Implement notification triggers. Test end-to-end with real messages.
Day 6	Polish & Bonus	Fix bugs. Improve UI. Add bonus features if time permits. Begin writing project report and recording demo video.
Day 7	Submission Day	Finalize report. Complete README. Confirm GitHub repo is accessible. Submit all deliverables before deadline.